



API Fundamentals Complete Guide

A comprehensive professor-style guide exploring definitions, architecture, protocols, and testing tools for modern developers



agenda

A structured roadmap through API concepts, architecture, protocols, and essential testing practices.

KEY TAKEAWAY

"By the end of this session, you'll understand how systems talk to each other and how to verify those conversations."

FOUNDATIONS

- What is an API?
- Common Types (REST, SOAP, GraphQL)
- Client-Server Architecture

SECURITY

- HTTP vs HTTPS
- Authentication & Authorization

MECHANICS

- API Protocols (HTTP, WebSocket)
- HTTP Methods & Response Codes
- Data Formats (JSON vs XML)

QA & TOOLS

- API Testing Strategies
- Testing Tools (Postman, Swagger)

WHAT IS AN API?

Application Programming Interface

A software intermediary that allows two applications to talk to each other. It defines a "contract" of inputs and outputs, hiding the internal complexity.

REAL-WORLD EXAMPLES



Weather Apps

Retrieves forecast data from remote servers



Social Login

"Sign in with Google" uses identity APIs



Payment Gateways

E-commerce sites processing cards via Stripe

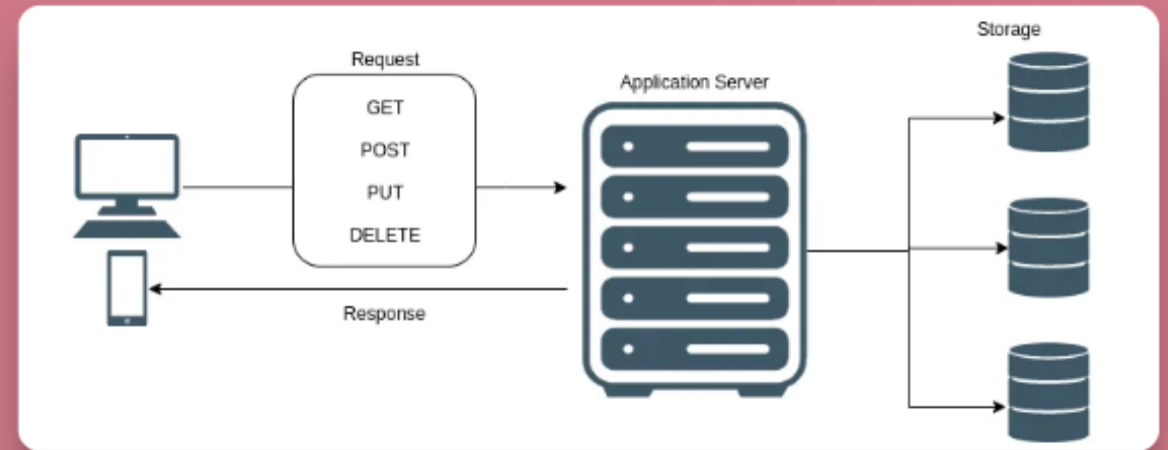
Request

`GET /weather?city=Paris`



Response (200 OK)

`{ "temp": 18, "unit": "C" }`



 Client-Server Architecture

COMMON TYPES OF API

Architectural Styles & Protocols

Different architectural styles suit different needs. While REST is the most common for web services, others like GraphQL and gRPC solve specific problems related to flexibility and performance.



PRO TIP

Choose the style based on your client's needs, not just popularity.



STANDARD

REST

Representational State Transfer

Resource-based, stateless, uses standard HTTP methods. Best for public web APIs & CRUD.



FLEXIBLE

GraphQL

Query Language for APIs

Client asks for exactly what it needs. Solves over-fetching. Great for complex frontends.



LEGACY/STRICT

SOAP

Simple Object Access Protocol

XML-based, strict contract (WSDL). ACID compliance. Common in enterprise & banking.



FAST

RPC / gRPC

Remote Procedure Call

Action-based. gRPC uses HTTP/2 + ProtoBuf. High performance for microservices.

API PROTOCOLS

Communication Standards

The protocol defines the rules for how data is exchanged. Choosing between standard HTTP, real-time WebSockets, or high-performance gRPC dramatically impacts your application's capabilities.

PROFESSOR'S NOTE

"90% of the time, HTTPS is what you need. Don't overengineer with gRPC unless you have specific microservices performance requirements."



HTTP / HTTPS

THE WEB STANDARD

Request-response model. Stateless and ubiquitous. HTTPS adds TLS encryption for security.

Best for: Public Web APIs, Mobile Backends



WebSocket

REAL-TIME BI-DIRECTIONAL

Persistent full-duplex connection. Allows server to push data to client without polling.

Best for: Chat apps, Live trading, Gaming



gRPC

HIGH PERFORMANCE RPC

Uses HTTP/2 and binary ProtoBuf data. Much faster than JSON but stricter contracts.

Best for: Internal microservices, Low-latency needs

HTTP METHODS

YOU NEED TO KNOW

The Verbs of the Web

HTTP methods define the action to be performed on a resource. Using the correct method is crucial for creating intuitive and standard-compliant RESTful APIs.



BEST PRACTICE

Use nouns in URL paths (e.g., `/users`) and let the HTTP method define the verb (action).



SAFE

GET

GET `/users`

Retrieve resources. Safe and idempotent (multiple calls have same result).



CREATE

POST

POST `/users`

Create a new resource. Not idempotent; creates new data each time.



REPLACE

PUT

PUT `/users/1`

Replace a resource entirely. Idempotent (sending same data twice is safe).



MODIFY

PATCH

PATCH `/users/1`

Partially update a resource. Sends only the data that changed.



DELETE

DESTRUCTIVE

DELETE `/users/1`

Remove a resource. Idempotent (deleting twice yields 404 the second time).



response codes

HTTP status codes act as the primary communication signal, instantly telling the client if their request succeeded, failed, or needs more action.

DEVELOPER TIP

"Don't just return 200 OK for errors. Using the correct 4xx or 5xx code is critical for monitoring tools and client-side logic."

2XX SUCCESS

- 200** OK (Standard success)
- 201** Created (New resource made)
- 204** No Content (Action successful)

3XX REDIRECTION

- 301** Moved Permanently
- 302** Found (Temporary redirect)
- 304** Not Modified (Cached)

4XX CLIENT ERROR

- 400** Bad Request (Invalid syntax)
- 401** Unauthorized (Auth required)
- 403** Forbidden (Access denied)
- 404** Not Found (Resource missing)

5XX SERVER ERROR

- 500** Internal Server Error
- 502** Bad Gateway
- 503** Service Unavailable

DATA FORMATS

Structured Information Exchange

APIs need a common language to structure data. While XML was the enterprise standard for decades, JSON has become the dominant format for modern web and mobile APIs due to its simplicity and native compatibility with JavaScript.

QUICK COMPARISON

JSON

Lightweight, readable, maps to objects.

XML

Verbose, supports schema validation, metadata.

JSON



JavaScript Object Notation

The modern standard. Used by REST and GraphQL APIs.

```
{  
  "id": 42,  
  "name": "Ava",  
  "tags": ["pro", "api"]  
}
```

XML



eXtensible Markup Language

Legacy and enterprise standard. Used by SOAP APIs.

```
<user id="42">  
  <name>Ava</name>  
  <tags>  
    <tag>pro</tag>  
    <tag>api</tag>  
  </tags>  
</user>
```

HTTP VS HTTPS

Securing the Web Layer

The key difference lies in **Encryption**. HTTPS adds a security layer (TLS/SSL) that encrypts data in transit, ensuring confidentiality and verifying the server's identity via certificates.

PROTOCOL SPECS

HTTP

Port 80
Plain Text Transfer

HTTPS

Port 443
Encrypted (TLS/SSL)

UNSECURE

HTTP

Vulnerable to eavesdropping and Man-in-the-Middle attacks.

Payload

Status: Visible

```
POST /login
user: "admin"
pass: "123456"
```

SECURE

HTTPS (HTTP + TLS)

Authenticity & Integrity via X.509 Certificates.

Payload

Status: Encrypted

```
POST /login
Encrypted: 7f8a9d...
[TLS Handshake Verified]
```

API AUTHENTICATION & AUTHORIZATION

Authentication verifies who you are, while **Authorization** determines what you can do. Both are critical for API security.



Identification

API Keys

Simple static string tokens. Best for server-to-server communication where low security is acceptable.



Delegation

OAuth 2.0

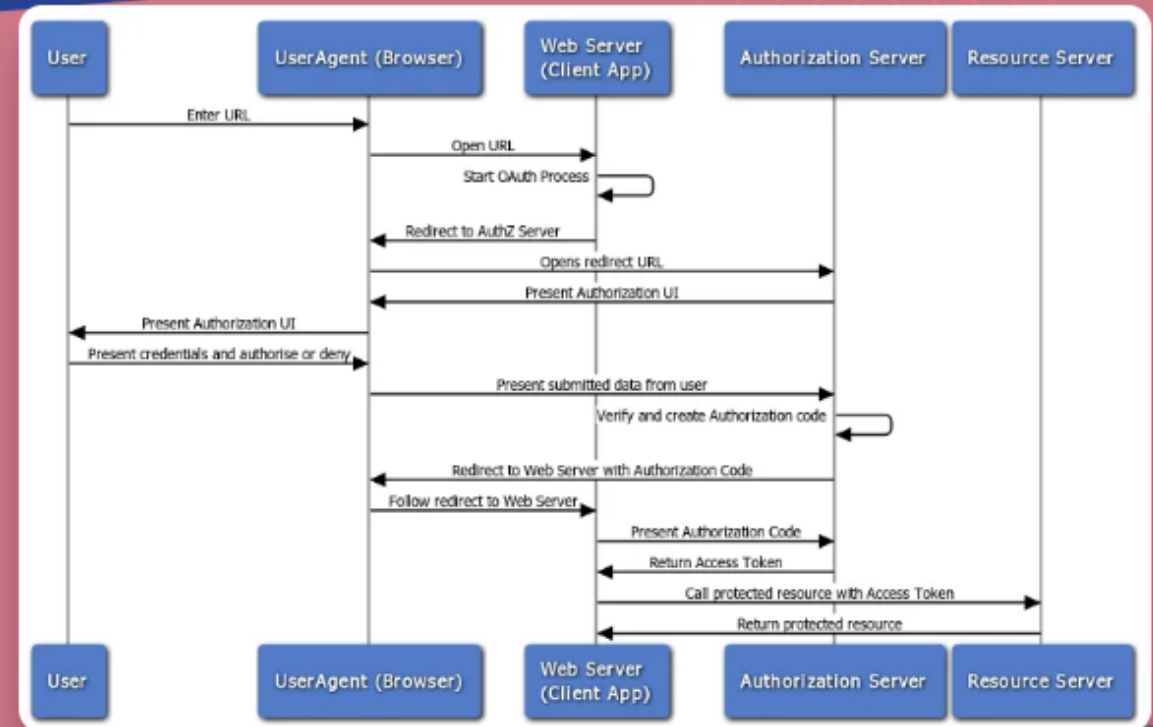
Industry standard for authorization. Grants access via tokens (scopes) without sharing user credentials.



Stateless

JWT (JSON Web Token)

Compact, URL-safe tokens containing signed claims. Ideal for stateless authentication in SPAs.



➡ OAuth 2.0 Access Flow



api testing

Ensuring reliability, security, and contract adherence before code reaches production.

WHY IT MATTERS

- ✓ Catch defects early
- ✓ Verify API contracts
- ✓ Ensure consistent performance

KEY TEST TYPES

- ⚙️ **Functional:** Verifies input/output against requirements.
- 🔗 **Integration:** Checks interaction between API and services/DB.
- ⌚ **Performance:** Tests latency, throughput, and load limits.

SCENARIOS

- 👍 **Happy Path:** Valid requests return expected data.
- ⚠️ **Negative Testing:** Invalid inputs return meaningful errors.
- 🛡️ **Security:** Unauthorized access is blocked (401/403).

WHAT TO VALIDATE

- 🔗 **Status Codes:** Correct 2xx, 4xx, 5xx usage.
- 📄 **Payloads:** Response body matches JSON schema.
- ⌚ **SLA:** Response time is within limits (e.g., <200ms).

EXAMPLE ASSERTIONS

- ✓ `response.status == 200`
- ✓ `body.user.id is Number`
- ✓ `time < 500ms`



testing tools

The right ecosystem of tools transforms API testing from a manual chore into a seamless, automated part of the development lifecycle.



Postman: The industry standard platform

POSTMAN

- ✓ Complete platform for API design, testing, and collaboration.
- ✓ Visual UI for sending requests and analyzing responses.

SWAGGER / OPENAPI

- 📄 Standard for defining API structure and documentation.
- 🔗 Enables contract testing and automatic mock server generation.

NEWMAN

- Command-line collection runner for Postman.
- 🔄 Enables integration of API tests into CI/CD pipelines (Jenkins, GitHub Actions).

OTHER TOOLS

- 🔗 **REST Assured:** Java library for testing RESTful web services.
- ⚡ **Insomnia:** Lightweight, fast REST and GraphQL client.